

안녕하세요.
프론트엔드 개발자 유상록입니다.

전화번호

010-8269-8150

이메일

dbtkdfhr2000@naver.com

GitHub

<https://github.com/dbtkdfhr>



About Me

협업을 통해 배우는 것을 좋아합니다.

배울 점은 누구에게나 존재합니다. 협업을 진행하며 새로운 지식을 습득하고 내 것으로 만들며 성장하고자 노력하고 있습니다.

안주하지 않습니다.

현재의 코드나 방식에 만족하지 않고, 항상 더 효율적인 구조와 성능 개선을 고민합니다.

새로운 지식을 배우는데 망설이지 않습니다

새로운 기술 트렌드에 호기심이 많으며, 배운 것을 주도적으로 프로젝트에 적용하고 팀과 공유합니다.

Education

국민대학교 소프트웨어학과

2019.03 ~ 2025.02 졸업

Project

수강신청 시스템

기술 스택: Java, Spring, Redis, MariaDB, WebFlux, R2DBC

개인 프로젝트

프로젝트 개요

수강신청 오픈 시 발생하는 고동시 요청을 안정적으로 처리하기 위한 REST API 서버입니다.

JPA + H2 기반의 단순한 구현에서 시작해, 각 단계에서 한계를 직접 측정하고 기술을 교체하며 최종 구조에 도달했습니다.

기술을 바꿀 때마다 Apache Bench와 Python concurrent.futures로 벤치마크를 수행하여 개선 효과를 수치로 검증했습니다.

비관적 락의 한계와 Redis 분산 락 도입

처음에는 JPA의 PESSIMISTIC_WRITE로 동시성을 제어했습니다.

단일 서버 환경에서는 정상적으로 동작했지만, 서버를 수평 확장하는 상황을 고려했을 때 DB 수준의 락은 서버 간에 공유되지 않는다는 근본적인 문제가 있었습니다.

또한 Tomcat의 기본 스레드 풀은 200개로 제한되어 있어, 수강신청 오픈 순간 5,000명이 동시에 접속하면 나머지 4,800명은 큐에 쌓이거나 타임아웃이 발생합니다.

이를 해결하기 위해 Redisson 기반의 Redis 분산 락을 도입했습니다.

락의 키를 lock:course:{id} 형태로 강좌 단위로 분리하여, 서로 다른 강좌에 대한 신청은 락 경쟁 없이 병렬 처리되도록 설계했습니다.

락 획득에 실패한 요청은 재시도 없이 즉시 409를 반환하여 락 대기로 인한 스레드 낭비를 방지했습니다.

Spring MVC에서 WebFlux로의 전환

Redis 분산 락을 적용하더라도 블로킹 스레드 모델 자체의 한계는 여전했습니다.

Tomcat은 요청 하나당 스레드 하나를 점유하는 구조이기 때문에, DB 응답을 기다리는 동안에도 스레드가 유휴 상태로 낭비됩니다.

이 문제를 해결하기 위해 Spring WebFlux + Netty 기반으로 전환했습니다.

Netty는 소수의 이벤트 루프 스레드가 수천 개의 커넥션을 비동기로 처리하기 때문에, 스레드 고갈 문제 자체가 발생하지 않습니다.

DB 드라이버도 JPA 대신 비동기 R2DBC로 교체하여 요청 처리 흐름 전체가 논블로킹으로 유지되도록 구성했습니다.

Project

H2에서 MariaDB로 전환 – 예상 밖의 결과와 재검증

WebFlux로 전환 후 H2 환경에서 벤치마크를 진행했을 때 예상과 다른 결과가 나왔습니다.

H2 읽기 부하 (동시 100, 3,000 요청)

지표	JPA	WebFlux
TPS	6457 req/s	3397 req/s
p95	30ms	126ms

JPA가 오히려 약 2배 빨랐습니다. H2는 인메모리 DB라 I/O 대기가 거의 없고, 이 상황에서는 WebFlux의 Reactor 체인과 Redis 왕복이 오히려 오버헤드로 작용했습니다.

WebFlux의 강점은 I/O 대기 시간이 길수록 극대화되는데,
H2 환경에서는 그 조건이 충족되지 않은 것입니다.

실제 환경에 가까운 측정을 위해 Docker로 MariaDB 11을 구성하고 동일한 벤치마크를 재수행했습니다.

MariaDB 읽기 부하

지표	JPA	WebFlux
TPS (동시 100)	1470 req/s	1786 req/s
TPS (동시 500)	2115 req/s	3076 req/s
p95 (동시 100)	120 ms	86ms
p95 (동시500)	407ms	293ms

실제 I/O가 발생하는 환경에서는 WebFlux가 읽기 성능에서도 앞섰고, 동시 접속이 많아질수록 격차가 더 벌어졌습니다.

Project

MariaDB 쓰기 동시성 (100명 동시, 정원 50명)

지표	JPA	WebFlux
성공 (201)	1건	50건
동시성 정확도	불안정	정확

JPA의 비관적 락이 MariaDB의 InnoDB 행 버전 관리와 충돌하여 HA_ERR_RECORD_CHANGED 오류가 대량으로 발생했고, 정원 50명 중 1명만 성공하는 결과가 나왔습니다.

H2 환경에서는 드러나지 않았던 문제가 실제 DB에서 검증된 것입니다.

Redis 분산 락은 MariaDB 환경에서도 정원 50명을 정확하게 제어했으며, JUnit으로 작성한 동시성 테스트(정원 1명 강좌에 100명 동시 신청 → 1명만 성공)도 통과했습니다.

Project

Schemafy – BFF 서버 설계 및 구현

기술 스택: NestJS, TypeScript, Axios, WebSocket, HMAC-SHA256

팀 프로젝트 (ERD 실시간 협업 툴)

프로젝트 개요

프론트엔드를 담당하던 중, 기존 구조의 보안 취약점을 발견하고 BFF 서버의 필요성을 팀에 제안하여 직접 설계 및 구현을 맡았습니다.

BFF 도입 배경 – 요청 출처를 신뢰할 수 없는 문제

기존 구조는 프론트엔드 클라이언트가 Spring Boot 백엔드에 직접 요청을 보내는 방식이었습니다. 백엔드에서 입력값 유효성 검사는 수행하지만, 유효성 조건을 만족하는 요청이라면 실제 클라이언트 애플리케이션에서 온 것인지, 악의적인 사용자가 직접 조작하여 보낸 것인지 구별할 방법이 없었습니다. 이를 해결하려면 백엔드가 검증된 출처의 요청만 신뢰하도록 만들어야 했고, HMAC 서명이 적합한 방법이었습니다.

그러나 HMAC 시크릿을 브라우저 클라이언트에 두면 번들 코드에서 탈취될 수 있어 서버 측에서만 보관해야 했습니다.

이 이유로 BFF를 중간 계층으로 두고, BFF만이 시크릿을 보유하여 백엔드로 나가는 모든 요청에 서명하는 구조를 설계했습니다.

HMAC 요청 서명 구현

BFF가 백엔드로 요청을 보낼 때, HTTP메서드 + 경로 + 타임스탬프 + Nonce +바디해시를 정규 문자열로 조합한 뒤 공유 시크릿으로 HMAC-SHA256 서명을 생성하여 X-Hmac-Signature, X-Hmac-Timestamp, X-Hmac-Nonce 헤더에 담았습니다.

백엔드는 동일한 방식으로 서명을 재계산해 요청이 BFF에서 온 것임을 검증합니다.

Nonce와 타임스탬프를 포함해 서명 재사용(Replay Attack)도 방지했습니다.

Axios 인터셉터에 서명 로직을 주입해 모든 요청에 자동으로 적용되도록 설계했습니다.

WebSocket 중계 – 인증 토큰 노출 방지

실시간 협업 기능의 WebSocket 연결도 동일한 문제가 있었습니다. 프론트엔드가 백엔드 WebSocket에 직접 연결하면 인증 토큰을 URL 쿼리스트링에 노출해야 하는 보안 문제가 발생합니다.

BFF의 WebSocket Gateway가 프론트엔드 연결을 받은 뒤, 쿠키에서 액세스 토큰을 추출하여 백엔드 WebSocket에 Cookie 헤더로 인증 정보를 담아 연결을 수립합니다.

이후 양방향 메시지를 투명하게 중계하는 브리지 구조로 설계했습니다.

Project

내부 오류 정보 은닉 – BFF 계층의 보안 이점

백엔드가 반환하는 에러 응답에는 내부 구현 상세, 스택 트레이스, 서버 구조를 유추할 수 있는 정보가 포함될 수 있습니다.

이를 프론트엔드에 그대로 전달하면 공격자가 내부 구조를 파악하는 데 활용할 수 있습니다.

BFF가 중간에서 모든 에러를 가로채 프로덕션 환경에서는 백엔드의 상세 오류 정보를 제거하고 클라이언트에게는 표준화된 오류 코드와 메시지만 전달합니다.

이처럼 BFF는 HMAC 서명으로 요청 출처를 검증하는 것뿐만 아니라, 응답 방향에서도 내부 정보가 외부로 유출되지 않도록 차단하는 보안 경계선 역할을 합니다.